



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Array e Lista

QXD0001 - Fundamentos de Programação

Prof. Bruno Góis Mateus (brunomateus@ufc.br)

Agenda

- Introdução
- Array em Go
- Percorrendo vetores
- Erros comuns
- Lista em Go

Introdução

Introdução

Por que precisamos de array?

- Imagine que você precisa escrever um programa para indicar a maior nota, a menor nota e a média dos alunos de uma turma de tamanho qualquer ?
- E se você precisasse saber quantos alunos tiraram a maior nota?



Precisamos
utilizar um
Array

Introdução

O que é um Array?

- Uma coleção de valores do mesmo tipo
- Permite guardar vários valores em uma única variável
- Os valores/elementos são armazenados de maneira ordenada
- Cada valor é identificado pelo seu índice

Introdução

Armazenamento em memória

- Ao criar um vetor, o computador reserva um bloco de endereços vizinhos

Memória RAM

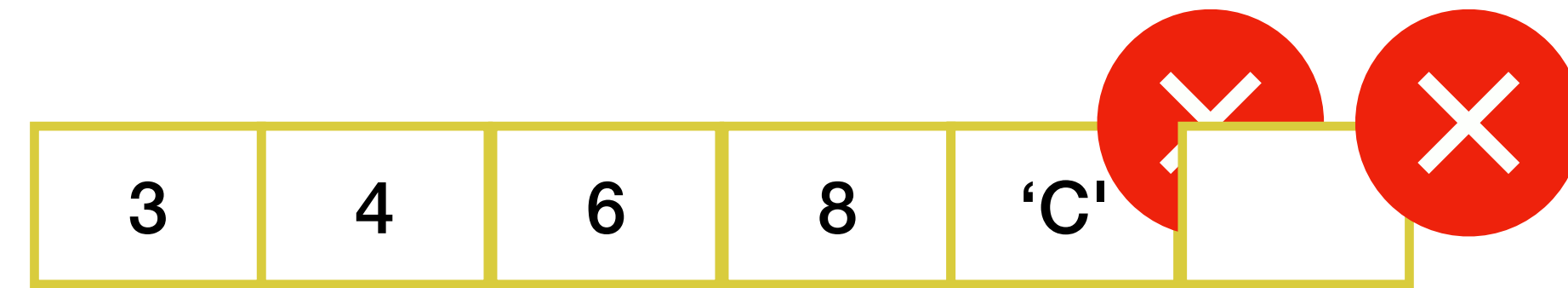
```
n := [5]float64{3, 4, 6, 8, 10}
```

		3	4	6	8	10	

Array em Go

Array em Go

- Homogêneo
 - Todos os elementos devem ser do mesmo tipo
- Tamanho Fixo
 - Uma vez criado com 10 posições, ele sempre terá 10 posições



Array em Go

- Indexação
 - O primeiro índice é sempre a de número 0
 - O último é $n-1$
- Acesso Direto
 - Você pode acessar qualquer posição instantaneamente se souber o índice

3	4	6	8
[0]	[1]	[2]	[3]

Array em Go

// Declaração

var notas **[5]float64** 

//Declarando.e inicializando

notas := **[5]int**{3, 4, 6, 8, 10} 

Array em Go

Acessando e alterando elementos

```
notas := [5]int{3, 4, 6, 8, 10}

// Acessando elementos - leitura
fmt.Println(notas[0])
fmt.Println(notas[2])

// Modificando um elemento - escrita
notas[1] = 10.0
```



3	10	6	8	10
[0]	[1]	[2]	[3]	[4]

Percorrendo vetores

Percorrendo vetores

A forma tradicional

```
notas := [3]float64{10, 8.5, 7.0}

for i := 0; i < len(notas); i++ {
    fmt.Println("Nota do aluno", i, "é", notas[i])
}
```

Retorna o tamanho do vetor

Vai receber o valor dos índices dos elementos do vetor

Percorrendo vetores

Usando o operador range

```
notas := [3]float64{10, 8.5, 7.0}
```

```
for indice, valor := range notas {  
    fmt.Println("Nota do aluno", indice, "é", valor)  
}
```

O valor do elemento atual, ou seja, aquele armazenado na posição indicada

Operador range

Vai receber o valor dos índices dos elementos do vetor

```
for _, valor := range notas {  
    fmt.Println(valor)  
}
```

Caso o índice não seja útil



**Em Go, quando
passamos um Array
para uma função, o Go
cria uma cópia
(passagem por valor)
de todo o array. Se o
array for gigantesco,
isso pode ser lento.**

Erros comuns

Erros comuns

✗ Índice fora do limite

```
notas := [5]int{3, 4, 6, 8, 10}
```

```
fmt.Println(notas[10])✗
```

Erros comuns

✗ Erro 2— Tipos diferentes

```
prova1 := [5]int{3, 4, 6, 8, 10}  
prova2 := [10]int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}  
  
var notas [5]int = prova2 ✗
```


Lista em Go

Lista em Go

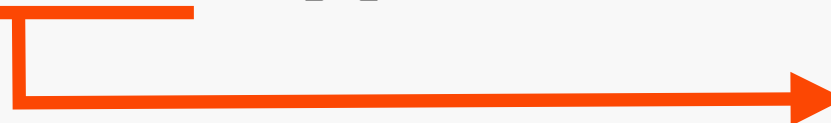
Diferença Visual na Sintaxe:

```
// Declarando um vetor
```

```
var notas [5]float64
```

```
//Declarando uma lista
```

```
var notas []float64  Tipo da lista
```

 Nome da variável

```
// Forma literal
```

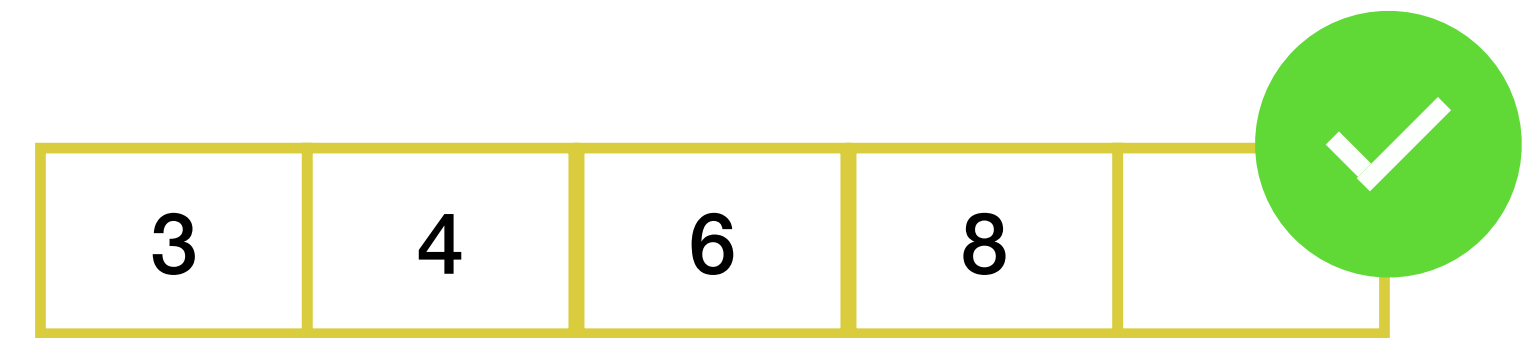
```
numeros := []int{10, 20, 30}
```



Por que usar Slices?
No Array clássico,
precisamos definir o
tamanho na hora de
compilar (ex: [5]int).
E se não tivermos
como prever?

Lista em Go

- São chamadas de Slices
 - Todos os elementos devem ser do mesmo tipo
 - Tamanho flexível
 - Pode ser alterado ao longo da execução



Lista em Go

Anatomia de um slice

- É uma estrutura que guarda 3 coisas:
 - **Ponteiro:**
 - Aponta para o primeiro elemento de um Array oculto na memória
 - **Tamanho (Length):**
 - Quantos elementos estão no Slice atualmente
 - **Capacidade (Capacity):**
 - O tamanho máximo que o Array oculto suporta antes de precisar crescer

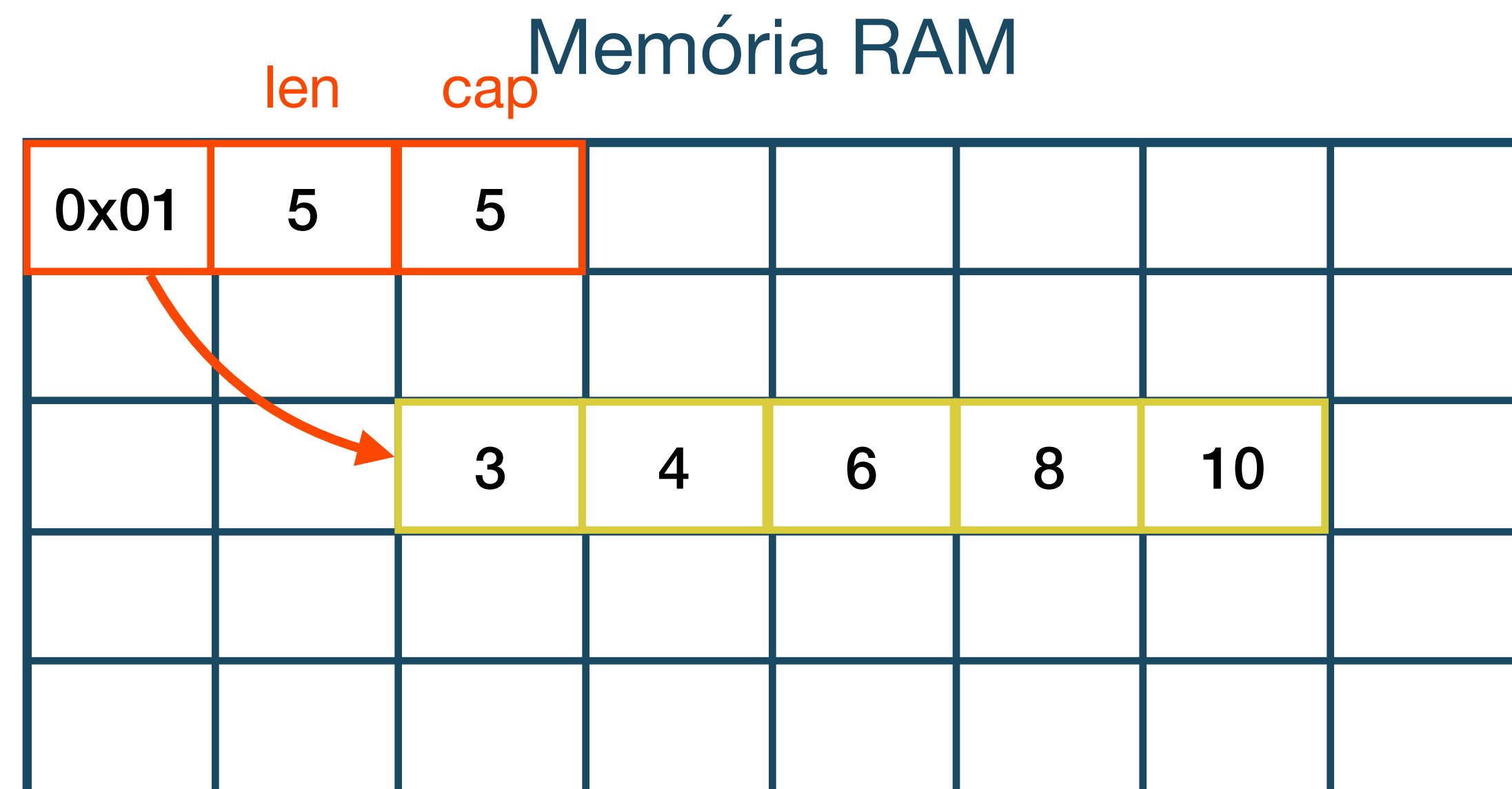
Lista em Go

Anatomia de um slice

```
n := []int{3, 4, 6, 8, 10}
```

```
// Tamanho de um slice  
fmt.Println(len(n))
```

```
// Capacidade de um slice  
fmt.Println(cap(n))
```





Como o Slice guarda o ponteiro, tamanho e capacidade, passá-lo para uma função é extremamente rápido e eficiente. Se a função alterarmos um elemento do Slice, essa alteração reflete no Slice original (Passagem por Referência implícita).

Lista em Go

Usando a função make

```
n := make([]int, 5)  
n := make([]int, 3, 5)
```

Memória RAM

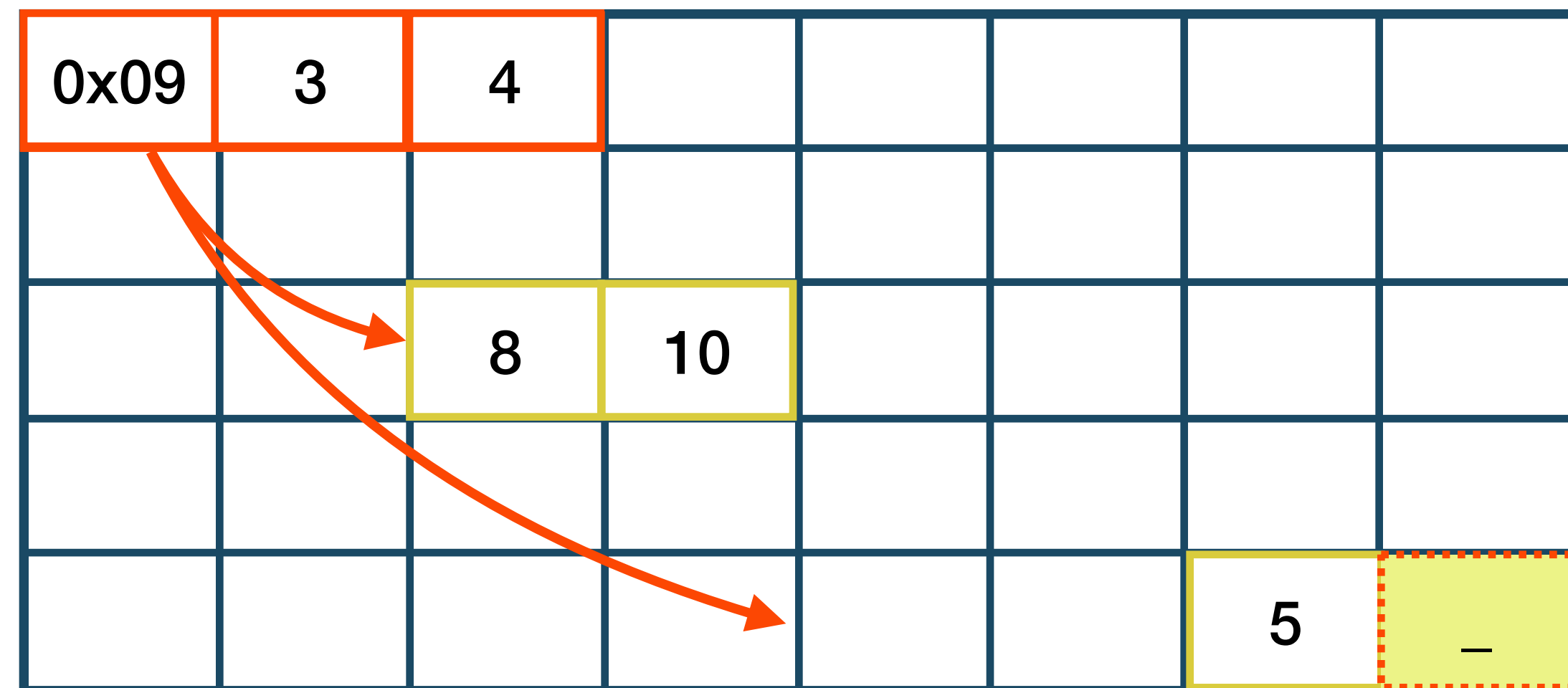
0x01	3	5					
		0	0	0	—	—	

Lista em Go

Alterando o tamanho de um slice

```
n := []int{8, 10}
n = append(n, 5)
```

Memória RAM



Lista em Go

Fatiamento

```
n := []int{3, 4, 6, 8, 10}
```

```
//Fatiamento - sintaxe
```

```
n[a : b]
```

Índice do primeiro elemento a NÃO ser incluído no novo slice

Índice do primeiro elemento a ser incluído no novo slice

```
//Podemos omitir valores
```

```
n[0:5]
```

```
n[:5]
```

```
n[0:]
```

```
n[:]
```

Lista em Go

Fatiamento

```
n := []int{3, 4, 6, 8, 10}
```

```
menores := n[:2] // n[0:2]
```

```
maiores := n[3:] // n[3:5]
```

// ⚠ Cuidado com slice de slice

```
menores[0] = 1
```

```
maiores[0] = 10
```

```
fmt.Println(n[0])
```

Memória RAM

0x01	5	5		0x01	2	5	
		1	4	6	10	10	
		0x03	2	2			



Cuidado ao Compartilhar Memória: Se você cria um `sliceB` fatiando um `sliceA`, ambos apontam para o mesmo `Array` na memória. **Alterar `sliceB` altera `sliceA`!**

A Solução: Se quiser uma cópia independente, use a função `copy()`.

Referências

- Go 101